Operating LLM-Generated Data Wrappers in Production: A 149-Day Case Study of What Breaks in a Smart-City Intake Layer

Anonymous Author(s)

***Abstract*—Every indicator a smart-city observatory shows needs a wrapper that pulls one series from one heterogeneous source and keeps pulling it as the source drifts, and the cost of writing and nursing those wrappers caps how many feeds an observatory sustains. We report a 149-day production case study of an intake layer that removes the per-source code: a non-technical user registers a source through a form, and the platform has a large language model write, run, and repair the extraction wrapper, mapping REST APIs, spreadsheets, and CSV files to a normalized $(\text{timestamp}, \text{value})$ series. Over 396 wrappers and 164,322 measurements, at least one wrapper in seven crashed with an uncaught exception; the repair loop stopped the crash in 42 of the 43 wrappers it reached and restored output in 32. What filled the error column was operational: 65 wrappers (16%) never received code because the model's free-tier quota, an overload, or our own restarts stopped generation, an execution watchdog we had set shorter than the feeds' polling period killed 69 API pollers, and one crash went unrepaired because the repair call itself hit the quota. Two of four inspected repairs recovered by swallowing an upstream error, a move our own generation prompt prescribes, so we score recovery as resumed output and treat value correctness as open; a re-extraction of eight public sources matched 84 of 95 stored values exactly, the rest upstream revisions. We close with the operational rules the deployment taught us, each of which cost us data or the means to audit it.**

***Index Terms*—LLM-generated code, automated program repair, data ingestion, smart-city observatory, autonomic computing, field failure study**

I. Introduction

A city run on evidence tracks many numbers that arrive in many forms: environmental readings from sensor APIs, visitor counts from tourism boards, energy and mobility figures from utilities and transport operators, each with its own endpoint or spreadsheet export, field names, date format, and language. None of it stands still: an indicator earns its place only while it stays current, so every feed must be pulled again and again, and every upstream change of format or endpoint absorbed without breaking the series already built. The usual way to take a source in is to write an extraction wrapper for it and then keep that wrapper alive as the source shifts underneath. This cost, paid once per source to build and then indefinitely to maintain, is why mediator-style integration has never reached past a small set of feeds [1]–[3], and it sets the ceiling on coverage: an observatory ends up showing whichever feeds someone found time to wire and to nurse, rather than the city.

Large language models (LLMs) can now write such wrappers from a description and a data sample, and can rewrite them from a stack trace. That makes a different intake layer possible, one where a domain user registers a source through a form and the platform generates, runs, and repairs the wrapper itself. The engineering question is not whether a model can write a wrapper, which benchmarks already answer [4], [5], but what breaks when hundreds of such wrappers run unattended for months, whether the failures indict the generated code or the environment, whether repair fixes faults or hides them, and what an operator is left holding. Those are dependability questions with no laboratory answer.

This paper is a case study that answers them for one deployment. The Observatory (anonymized) is a smart-city observatory for a European region whose intake layer is built from LLM-generated wrappers. Each wrapper is an *autonomic* element in the sense of Kephart and Chess [6], managed by a monitor–analyze–plan–execute loop over shared knowledge (MAPE-K) [7], [8] whose actuator is a model that writes code rather than a parameter tuner; the loop organizes the exposition and names the boundary of what it can fix (Section II places it against prior repair work). The adaptivity is bounded: the platform adapts to a source's format and transport within the structured tabular family, REST/JSON APIs, spreadsheets, and delimited files, all projected onto one univariate $(\text{timestamp}, \text{value})$ series.

The contributions are the architecture and its one narrow contract (Section III) and, mainly, the empirical findings from the 149-day run. First, the *binding faults are operational, not generated-code defects*: 65 wrappers never received code because generation failed on quota, overload, or a platform restart, 69 API pollers were killed by a watchdog we configured shorter than their polling period, and the one crashed wrapper whose code went unrepaired was blocked by that same quota (Section V-C). Second, *cheap generation decouples attempt from yield*: only 284 of 396 wrappers ever produced data, and most of the rest failed loudly on quota and were deleted by their users, shifting the bottleneck from authoring to operations (Section V-A). Third, *crash-triggered repair can mask faults rather than fix them*, so we score recovery as resumed output, report the exposure time over which repaired wrappers were observed, and treat value correctness

as open (Sections V-B–V-D). We add a bounded estimate of what generation cost and a re-extraction consistency check on the public sources still reachable (Section V-E). The analysis script and a derived event dataset accompany the paper.[1]

II. Related Work

The smart-tourism literature argues for destination-scale data infrastructure [9]–[11], while field reports describe patchy uptake, dashboards assembled from warehouses and surveys, and getting the data in at all as the binding constraint on sustained monitoring [12]–[15]; these accounts say little about how intake is built.

Autonomic computing proposed self-managing components with closed control loops [6], structured as MAPE-K [7] and cataloged by self-* property [8]; a recent roadmap surveys generative models as actuators in self-adaptive systems [16].

Language models write runnable programs [17] and repair them [18] through self-critique, reflection, or execution feedback [19]–[21], with inconsistent benefit [22]; repair has run in production at industrial scale [23], [24] and autonomously [25]. On the data side, Evaporate synthesizes extraction code [4], AutoScraper builds reusable scrapers [5], agents are benchmarked on enterprise data workflows [26], prompted and fine-tuned models handle preprocessing [27], [28], and detecting silent pipeline degradation is a line of work our masking finding motivates [29]. The loop itself is not new: the repair is LLM program repair, the generation Evaporate-style synthesis, the probing ordinary tool use. What those systems do not provide, and what we contribute, is a measured population of *field* faults from a live deployment, in the tradition of field-failure studies that found operator and configuration faults, not code, behind most service outages [30], [31], and of the finding that swallowed exceptions and empty error handlers sit behind many production failures [32].

Wrapper induction derived extraction rules from labeled examples [2], [33]–[35], and keeping a wrapper working as its source drifts was a distinct problem from the start [3], [36]. Extract-transform-load (ETL) pipelines have mostly been designed by hand [37], [38], with automation reaching schema matching [39], curation [40], and wrangling [41]. Dataspaces [42], polystores [43], and schema-on-read lakes make data source-agnostic for the *consumer*; we work the producer side.

III. Architecture and the Fixed Time-Series Contract

The Observatory runs as a set of containerized services, user, knowledge-base, data-collector, resource, and indicator (all Python and FastAPI), reached through an nginx gateway with a React dashboard in front. Control traffic and data traffic ride separate RabbitMQ brokers, each service keeps its own datastore, and an optional Keycloak provider handles single sign-on. Fig. 1 lays out the components and the two planes that join them: the ingestion plane, along which wrapper processes read the external sources and emit points through the data broker to the collector and on to the indicator store, and the REST serving plane through the gateway to the dashboard.

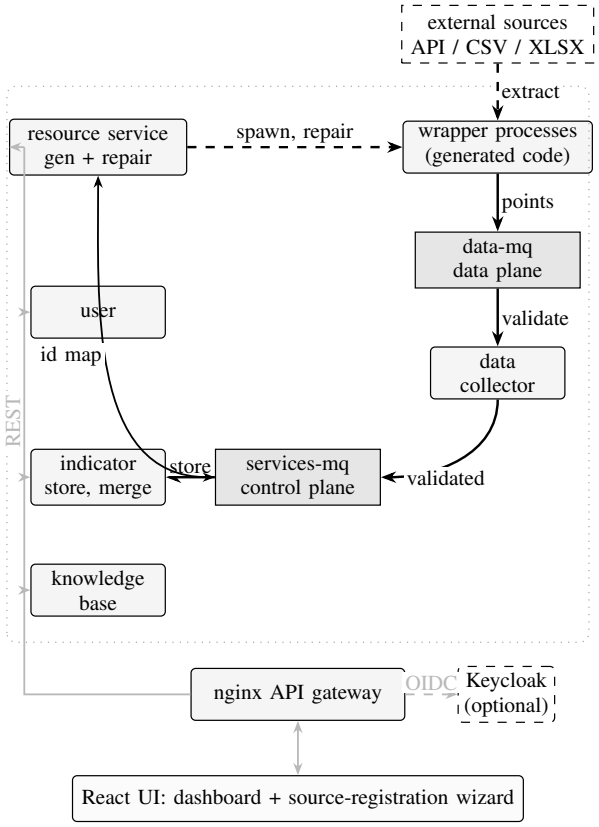


Fig. 1. Whole-system architecture. The ingestion path runs down the right-hand column: wrapper processes spawned and repaired by the resource service (dashed) read the external sources and push points through the data-plane broker and the collector (solid black) to the indicator store. The grey spine on the left is the REST serving plane, through which the nginx gateway fronts the Python/FastAPI services for the React dashboard; Keycloak provides optional OpenID Connect (OIDC) sign-on.

Format-adaptive intake rests on one thin interface: whatever its format, a wrapper emits point batches $\{(x, y)\}$ with x an ISO 8601 timestamp and y a float; the code above that interface is generated per source, everything below it is shared plumbing, and a source is in scope precisely when it projects onto one such series. Out of scope by construction are unstructured content, push and streaming transports, multivariate, geospatial, and graph data, and anything that needs a join across feeds; the unrepairable single-column CSV of Section V-D is this boundary made concrete. Four nouns recur below. A *source* is the external endpoint or file; a *resource* is the platform record a user creates for it through the web wizard (a file upload, or an API endpoint and its credentials, plus the indicator's metadata); a *wrapper* is the generated process the resource service owns for that resource; and an *indicator* is the served series the collector feeds after validating each batch against the contract. Deleting a resource stops

[1]For review, the analysis scripts, the derived per-wrapper and per-repair event dataset, and the prompt templates are mirrored anonymously at https://anonymous.4open.science/r/observatory-prdc; the platform's service code is released with the camera-ready.

and orphans its wrapper; it leaves `completed` in place but overwrites `error` with `stopped`. Source type is one argument to a single generation routine, not a code path of its own.

IV. One Manager per Wrapper

Each wrapper is the element a single autonomic manager looks after: the generated code plus its watermarks, retry counter, and status. The manager watches process state, classifies how a process ended, plans a fresh or corrected artifact, and deploys it, against a shared store of knowledge. Some of that knowledge is fixed: the wrapper template, which encodes the Section III contract as code, and a standing list of hard rules and formatting notes distilled from earlier failures (leave the backfill driver alone, invent no data, treat date ranges as inclusive, never raise on an HTTP error but return an empty batch). The rest is per-wrapper and changes at runtime: a pair of watermarks (earliest and latest timestamps sent), a retry counter capped at three, a timestamped execution log, and the stored wrapper document with its metadata, source configuration, current code, and status.

A. Generation

The actuator is Google's Gemini Flash, reached through the provider's Python SDK (software development kit) at default decoding settings and bound to the floating `gemini-flash-latest` alias. It does not write code freely but fills a fixed wrapper class: only extraction (`fetch_external_data`) and, for APIs, a windowed fetch and a polling period are open to it, while the backfill driver, transport, batching, and run loop are library code the prompt tells it not to touch. A template engine assembles the prompt from the indicator metadata, the source configuration, a brief data sample (20 CSV lines, 15 rows per sheet, or a captured API reply), the required output shape, the wrapper template, and the rule list. For an API the same call gives the model a sandboxed `fetch_url` tool returning the first 2500 characters of a live reply, up to ten calls, so it learns the response's shape rather than guessing it; the service builds authentication headers, so the model never handles credentials. A static check gates deployment (the file parses, no placeholder is left); a first failure earns one corrective prompt, a second sends the wrapper to `error`. Nothing is executed as a test, so any runtime fault is the repair loop's problem.

B. Deployment

Deployment is ordinary process supervision. The resource service runs each wrapper as its own operating-system process under a concurrency semaphore (default 100) and a per-run watchdog (3600 s at the pinned commit; Section V-C dates its effect); a file wrapper runs once, an API wrapper back-fills history in batches and then polls at the period its periodicity implies; a monitor reads process state every 30 s; after a restart a wrapper resumes from its high watermark, so a relaunch neither duplicates nor skips points.

C. Repair

The 30 s monitor reads each process's exit status. An abnormal exit, in practice always code 1, is separated from a watchdog kill and from a provider error during generation, and only the first becomes a repair. The plan is one model call carrying the full crashed source, its captured stderr, and the final twenty lines of stdout, with the instruction to return a complete corrected file that preserves the class and addresses that traceback; for an API wrapper it may re-interrogate the endpoint through the same `fetch_url` tool, under the same cap. The returned file passes the same static check, replaces the old one, bumps the retry counter, and the process is relaunched from its watermarks. At most three repairs are granted; the fourth-time loser stays in `error` for a person. The trigger is a live exit code, so the feedback carries the source's real behavior, and the plan re-emits the whole wrapper [21], [22].

D. Scope of control

The same view marks the limit of the manager's reach, drawn before looking at the counts: a fault is *inside* the manager when a change to the wrapper's code can fix it, and *outside* when the fix lies in something the code cannot rewrite (a spent quota, a stale credential, a watchdog shorter than the polling period). The boundary is a design choice: watchdog and polling period are both platform values, so a cadence mismatch is outside only because we left interval-setting outside.

V. Production Case Study

The figures come from a 149-day run (January 13, 2026 to June 11, 2026) captured as one complete state snapshot: the wrapper collection (396 documents with generated code and execution logs), the per-wrapper logs, the resource and indicator stores, the service logs, and container statistics. Every number below is computed from that snapshot by the released script, except the priced cost estimate and the live re-extraction of Section V-E, which also draw on the prompt templates, published prices, and the sources as of September 2026.

A. Generation and yield

The manager produced 396 wrappers, 188 for spreadsheet sources, 184 for APIs, and 24 for CSV, over three domains (environment 203, economy 139, sociocultural 54); between them they collected 164,322 points (53,530 from spreadsheets, 79,411 from APIs, 31,381 from CSV). Volume is long-tailed (median 27.5 points per wrapper, 90th percentile 967), and 284 of 396 sent at least one. Generation request to first running process took a median 26.8 s (90th percentile 109.8 s, $n = 331$), queueing and static check included. Downstream, 111 of the 179 active indicators carry data through the whole pipeline. Cheap generation comes with a long silent tail: 112 wrappers delivered no data: 69 deleted by their users, 51 of them after generation had failed on quota or overload and 18 after running empty; 14 generation failures still in `error`; 6 watchdog-killed pollers that emitted nothing; 22 that completed yet found nothing usable; and 1 still executing. Most of that tail failed

TABLE I
PER-SOURCE-TYPE OUTCOMES OVER THE 149-DAY RUN. *Resumed* COUNTS REPAIRED WRAPPERS THAT SENT DATA AGAIN AFTER THEIR LAST REPAIR (32 OF 43, SECTION V-B).

Type	n	Completed	Repaired	Resumed	Points
XLSX	188	104 (55.3%)	26	24	53,530
API	184	52 (28.3%)[a]	14	7	79,411
CSV	24	14 (58.3%)	3	1	31,381
All	396	170 (42.9%)	43	32	164,322

[a]A bookkeeping state for API pollers: the operator's re-execute route marks it at launch and the supervisor does not respawn completed wrappers, so all 52 ended silently on a later deploy; none finished on its own (Section V-C).

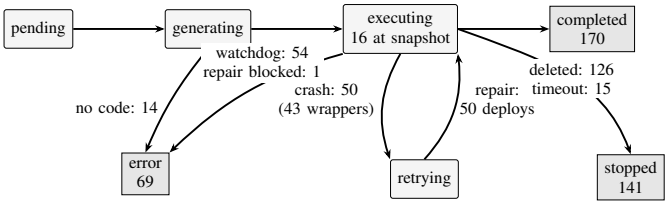


Fig. 2. Status machine, annotated with production counts ($n = 396$). The crash-and-repair branch touched 43 wrappers across 50 redeployments; most `error` outcomes are watchdog kills of long-cadence API pollers rather than faulty code (Section V-C).

loudly and was cleaned up; the 22 clean exits with nothing are the silent-failure class only value-level monitoring can catch. Counted by source rather than by wrapper document, the 396 wrappers cover 246 distinct sources, 85 of them registered more than once (mostly re-creations after a failed attempt), and 205 sources (83%) produced data; 37 wrappers whose name marks them as tests account for 51,864 points, a third of the total. Outcomes by source type appear in Table I, and the status machine with production counts in Fig. 2, where the 15 "timeout" entries into `stopped` are watchdog-killed pollers whose resource a user later deleted.

B. Repair effectiveness and exposure

43 wrappers (10.9%; 26 spreadsheet, 14 API, 3 CSV) threw an uncaught exception while running and received at least one repair, 50 redeployments in all; a 44th crashed wrapper never received a repair because the repair call itself hit the model quota, and Section V-C counts it. Repair depth was 37 wrappers at one attempt, 5 at two, and 1 at three. "Recovery" can be read three ways that do not agree, so we keep them apart and rank them. The weakest reading is recurrence: the same crash came back after the last repair for exactly one of the 43, so 42 stayed crash-free. Final status is the next: 39 of 43 (90.7%) closed in a non-error state, a count that still includes the one wrapper that burned all three attempts and shows non-error only because its resource was deleted afterwards, which we do not score as a win. The reading we adopt as primary is resumed output: 32 of 43 sent data again after their last repair, 20,351 points in all, about an eighth of everything collected, though one test feed accounts for 17,466 of them. Repairs were fast and untouched by a person: a median 33.4 s from crash to redeployment (quartiles 24.2 and 50.8 s, 90th percentile 60.5 s, longest 80.4 s), up to 30 s of which is the detection cycle. The typical crash arrived 16.8 s into a run, in the fetch-and-parse stage.

A wrapper observed only briefly after its repair has had little chance to fail again, so recurrence and resumed output are only as strong as the exposure behind them. We therefore measured, for each repaired wrapper, the time from its last repair to its last evidence of life (last log entry, emitted point, or stored segment; the snapshot for the three still executing). The 29 repaired file wrappers run once and exit, so for them "no recurrence" means only that the repaired run completed; their exposure is not informative. The 14 repaired API pollers are the real test: they were observed for a median 16.6 days after repair (quartiles 8.7 and 28.0, maximum 30.8, 230 wrapper-days in all) and survived 14 platform relaunch events (13 service restarts and one boot loop, below) on 10 dates without a crash. Nineteen of the 43 later had their resource deleted, eight of them as the event that ended observation, so for those the recovery is unverified rather than confirmed.

Two features of the timeline qualify the counts. For the first 65 days no repair loop was active (it first appears in the logs on 2026-03-19); 36 crashes of the same exit class went unrepaired in that stretch, and once the loop was on it repaired 24 of those 36 wrappers, most in the mass relaunch below. And 31 of the 50 repair events land on a single day, when a credential fix set off a redeploy that restarted every wrapper together, hours after a service boot loop had cycled 623 times in 3.4 hours and respawned about 40 pollers per cycle (20,849 starts, recorded only by the wrappers' own log entries), so the events are correlated rather than 50 independent failures; restricted to each wrapper's first repair, the 17 that first crashed on other days have a median 34.0 s (range 15.5–80.4) against 33.2 s for the 26-wrapper batch, and 15 of those 17 ended non-error. Because each relaunch overwrites stderr, a triggering trace survives only where the last run also crashed; for recovered wrappers we read the cause from the fixes their code carries, so these counts describe the model's *repairs*, not independently observed causes.

C. Faults inside and outside the manager

This is the paper's central result, and it has two halves. Uncaught exceptions were not rare: at least 56 wrappers, one in seven, exited with one at some point (the count is a lower bound, since regeneration erases a wrapper's earlier log); 44 of these were code faults the loop could address (43 repaired, one quota-blocked) and 12 cleared on a later relaunch with no code change, so were transient. The loop stopped the crash in 42 of the 43 it reached, so nearly every wrapper that ended in `error` did so for an operational reason. Table II sorts the 69 wrappers in `error` by cause, and one of them indicts the generated code. Fourteen never reached the running stage: ten ran into the model's free-tier request quota, one a provider overload, and three were cut off mid-generation when a service restarted. The same status filter hides 51 more: every one of the 65 wrappers without stored code is a generation failure (56

TABLE II
WHY THE 69 ERROR-STATUS WRAPPERS ENDED THERE, AND HOW THE REPAIR LOOP FARED OVER THE 43 CRASHED WRAPPERS AND THEIR 50 AUTOMATIC REPAIRS.

(a) Root causes of `error` status ($n = 69$)		
Stage	Root cause	n
Generation	Model free-tier request quota (HTTP 429)[c]	10
Generation	Model provider overloaded (HTTP 503)	1
Generation	Service restart during generation	3
Runtime	Execution watchdog (3600 s), API pollers[a]	54
Runtime	Crash; repair blocked by model quota (429)	1
	Total	69
(b) Repair outcomes (43 repaired wrappers, all crashes exit 1)		
Repaired once, crash never recurred		37
Repaired twice, crash never recurred		5
Still crashing after 3 repairs (retries exhausted)		1
Final status: completed / stopped / executing		28 / 8 / 3
Final status: error (later watchdog timeout)[b]		4
Crash-free after repair (recurrence)		42/43
Median crash-to-redeploy repair latency		33.4 s

[a]Marked by the watchdog's termination message; stderr held no traceback (53 of 54 blank, one harmless warning). See Section V-C for their coverage cost. [b]Each of the four was repaired and never crashed again; the later `error` comes from the watchdog limit, not the repaired code. [c]Another 51 generation failures (46 quota, 3 overload, 1 restart, 1 unrecorded) were deleted by their users and appear as `stopped`.

on quota, 4 on overload or restart, 1 unrecorded), and 51 of them were deleted by their users before the snapshot, so 16% of everything registered never got as far as running code. The larger set of 54 are API pollers the 3600 s watchdog stopped between polling cycles, 53 of them leaving no traceback. These are configuration faults of our own making: a platform execution timeout shorter than the feeds' daily or monthly rhythm, marked in the logs by the watchdog's own termination message and an empty stderr. Fifteen more pollers were killed the same way and later deleted by their users, so they appear as `stopped` (Fig. 2); 69 of 184 API wrappers (38%) were killed in all, 64 of them one hour after the deploy that introduced the timeout, in the first monitor tick past the limit, and they had shipped 61,627 points, 78% of all API points. Read from the generated code rather than the user's label, 34 of the 69 polled daily, 3 monthly, 26 yearly or slower, and 6 hourly-or-faster pollers were killed while idle past the hour: while the limit was in force, no long-cadence API feed survived its first idle hour. No kill is recorded after 2026-05-21, and the 16 pollers running at the snapshot had been up for eight hours, so the limit was evidently relaxed late in the run; the code at the pinned commit still defaults to 3600 s. A third operational class hides under `completed`: the operator's re-execute route marks an API wrapper completed at launch, and the supervisor does not respawn completed wrappers after a restart, so all 52 API wrappers in that state (24 daily, 27 yearly or slower) ended silently on a later deploy; none finished on its own. One wrapper crashed and could not be mended because the repair call itself struck the quota ceiling, the lone error-status case where generated code went unrepaired. The lesson is about the workload and the loop together: code defects happen at the rate generation makes them, the loop retires them at machine speed, and what is left on the operator's desk is the envelope of quota, credentials, service availability, and cadence-to-timeout matching. The specific watchdog and free-tier quota are this deployment's instantiation; the category is what we expect to transfer. The manager removes the per-source *code*, not every per-source decision.

D. Repair case studies

Four wrappers, chosen to exercise the boundary rather than sampled at random, put faces on the partition. A yearly urban-waste API wrapper crashed 82 s into its backfill on a timezone-aware-versus-naive comparison; the repair normalizes both bounds and downgrades an empty fetch to a log line (34.7 s, 596 points). An air-quality wrapper whose URL already pinned a date window had a second one appended and got an HTTP 400; the repair strips the duplicate and swallows any residual 400 (51.3 s, 395 points, still streaming). A workbook repeated its localized year header among the values, so the wrapper died converting it to an integer; the repair coerces to numeric and drops non-numeric rows, as do three siblings (61.6 s). The out-of-scope case is a single-column export: three repairs tried delimiter tricks but could not invent a $(\text{time}, \text{value})$ pair, while a two-column sibling was repaired at once. Two of the four "recover" by absorbing an upstream error rather than extracting the intended data, which is exactly the move that can mask data loss behind a clean exit; both follow a rule in our own generation prompt (never raise on an HTTP error, return an empty batch), so the masking is policy we wrote, executed faithfully. The manager mends code, and it cannot supply data the source withholds. Pre-repair code is not retained, so the masking rate cannot be measured by diff; a static scan of the final code of the 43 repaired wrappers against the 288 never-repaired ones with code finds the swallow-and-continue and coercion patterns equally common in both (they come from the template), with one exception: a bare `except` clause appears in six repaired wrappers (seven clauses) and in none of the controls.

E. Cost, and a consistency check on values

The platform logged no token counts, and the deployment ran on the provider's free tier, so its model spend was zero and the binding limit was a daily request quota rather than money. What the run would have cost at paid rates is an estimate we label as such. From the prompt templates at the deployed commit and the 331 wrappers with stored code we measured the fixed prompt (4,331 tokens for an API source, about 3,200–3,300 for files, at four characters per token), the variable sample and metadata, and the generated code (median 2,520 output tokens), priced at the rates published on 2026-09-17 for the model the alias resolved to (0.50 and 3.00 USD per million input and output tokens). A generation then costs about 0.010 USD at the median and a repair, which carries the crashed source and its trace, about 0.011 USD; the 396 generation attempts and 50 repairs together fall between 4.5

USD with no tool calls and 21 USD if every API wrapper spent its full budget of probing calls, about three times that at the successor model's rates, plus whatever unlogged reasoning tokens added. The estimate omits the corrective prompt a failed static check earns, prices the last fifteen days at the earlier model's rate, and treats probing calls as flat context; no hand-written wrapper exists for comparison. What the estimate does establish is the price of the choice we made: the whole run would have cost between 1.6 and 7.4 cents per data-yielding wrapper at paid rates, so the 57 quota faults, 14% of everything registered, were avoidable for less than an engineer-hour.

The collector checks each batch's shape, not whether the values match the source, and the snapshot retains no copy of the sources, so value correctness could not be verified from the dump. As a bounded substitute we took the stored code of eight API wrappers whose endpoints are public and need no credential (seven from a national statistics portal, one weather feed; six of the eight had been repaired), re-ran only their extraction logic against the live source three months after the snapshot with at most three requests each, and compared the fresh $(\text{timestamp}, \text{value})$ pairs with the dumped points over the overlapping range; this is a consistency check on stable public sources, since the same code re-run against the same endpoint cannot reveal a wrong column or unit. All eight sources were still reachable; six wrappers yielded 95 overlapping timestamps (77 of them from the two never-repaired wrappers, 18 from repaired code), of which 84 matched exactly and 11 differed, every difference an upstream revision of a published statistic (one annual value revised by 0.2%, ten monthly values revised down by up to 1.9%), so none is an extraction error. Four wrappers matched on every overlapping point; two were inconclusive (a deleted resource with no stored segments, and a request budget spent outside the stored range). This is a small, convenience-sampled check on the sources that happen to be public and stable, and it leaves spreadsheets, where 26 of the 43 crashes occurred, unchecked.

F. Pipeline integrity

Of the 164,322 points emitted, 148,131 reach the resource store and 84,431 survive consolidation and merging; the attrition is bookkeeping (deleted resources, regeneration, deduplication), not loss. Over 30 days the collector rejected nothing on schema grounds, all 153 control-plane messages in an 8.3-hour window arrived, and a container rebuild before the snapshot restarted all 16 running wrappers, 15 of which resumed from their watermarks.

VI. Lessons and Threats to Validity

A. Lessons for operators

Each of the following cost us data or the means to audit it, and each is a rule the platform could enforce rather than a habit. (1) Derive the execution timeout from the generated polling period, or make the watchdog idle-aware: a fixed 3600 s limit killed every daily, monthly, and yearly API poller within one monitor tick of its first idle hour, 64 on the day it shipped. (2) Alert on clean exits that emit nothing and on pollers that stop being respawned: 22 wrappers completed with zero points and 52 API pollers were orphaned by a bookkeeping status, and nothing distinguished either from a healthy run. (3) Treat the model quota as a dependency with a budget: it stopped 57 wrappers, 14% of everything registered, for want of a few dollars. (4) Retain stderr and the source sample across relaunches: the relaunch overwrote the trace for 42 of the 43 repaired wrappers and no copy of the source exists, so neither the cause nor the value can be audited afterwards. (5) Pin the model version: the alias resolved to two models during the run. (6) Monitor values, not exits: repair can mask data loss behind a clean status, a trust hazard when the series inform policy, so the intended data, not the process, is what needs a health signal. (7) Guard the supervisor against restart storms: the boot loop of Section V-B killed its children silently 623 times in an afternoon, and only the wrappers' start entries record it. Beyond these, cheap generation lets an observatory attempt the long tail of feeds it once skipped, if it budgets for triaging the attempts that never yield.

B. Threats to validity

Internal. The evidence is observational, from one deployment with no side-by-side control; the pre-automation period is a reference, not a matched arm, the crash events are correlated, and for repaired wrappers the cause is read from the fix because the relaunch destroyed the trace. Root causes were assigned by the authors from logs without a second rater. *Construct.* Recovery is scored as resumed output, and the exposure analysis shows how much observation backs that score; the collector inspects shape and never truth, and the re-extraction check covers only public, stable sources, mostly through unrepaired code. The cost figure is a counterfactual estimate. *External.* One model family, one region, a user-selected source mix, and a rule list that accreted over the run, so the code-defect rate is partly co-produced by human curation; the alias resolved to one model until 2026-05-20 and to its successor by 2026-05-27, so the numbers pool two models. We do not ablate probing, the template, or whole-file repair. *Security.* Model-written code runs as live processes contained only by process isolation, a semaphore, and the watchdog, and the generation prompt ingests a live source sample, so an injected instruction in a CSV or API reply could steer code that runs with the resource's credentials; the static check catches unfilled placeholders, not intent, and an egress allow-list plus a filesystem-free sandbox are the mitigations we would add.

VII. Conclusion

We operated LLM-generated extraction wrappers as live infrastructure for 149 days: 396 wrappers, 164,322 points, at least one wrapper in seven crashing at some point, and the repair loop stopping the crash in 42 of the 43 it reached and restoring output in 32, at a median 33.4 s. The faults that filled the error column were operational, a timeout we set, a quota we exhausted, a service we restarted; what stayed unresolved was smaller and harder: 22 wrappers that exited clean with

nothing, 52 pollers orphaned by a status, one crash the loop could not mend, and one it never reached. What the loop could not absorb sets the agenda: checking extracted values for meaning, and folding cadence, quota, and credential faults into the manager as state it owns.

References

[1] G. Wiederhold, "Mediators in the architecture of future information systems," *Computer*, vol. 25, no. 3, pp. 38–49, 1992.

[2] N. Kushmerick, "Wrapper induction: Efficiency and expressiveness," *Artificial Intelligence*, vol. 118, no. 1–2, pp. 15–68, 2000.

[3] K. Lerman, S. N. Minton, and C. A. Knoblock, "Wrapper maintenance: A machine learning approach," *Journal of Artificial Intelligence Research*, vol. 18, pp. 149–181, 2003.

[4] S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language models enable simple systems for generating structured views of heterogeneous data lakes," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 92–105, 2023.

[5] W. Huang, Z. Gu, C. Peng, J. Liang, Z. Li, Y. Xiao, L. Wen, and Z. Chen, "AutoScraper: A progressive understanding web agent for web scraper generation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 2371–2389. [Online]. Available: https://aclanthology.org/2024.emnlp-main.141/

[6] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[7] IBM Corporation, "An architectural blueprint for autonomic computing," IBM, White Paper, Jun. 2006.

[8] M. C. Huebscher and J. A. McCann, "A survey of autonomic computing—degrees, models, and applications," *ACM Computing Surveys*, vol. 40, no. 3, pp. 7:1–7:28, 2008.

[9] U. Gretzel, M. Sigala, Z. Xiang, and C. Koo, "Smart tourism: foundations and developments," *Electronic Markets*, vol. 25, no. 3, pp. 179–188, 2015.

[10] D. Buhalis and A. Amaranggana, "Smart tourism destinations," in *Information and Communication Technologies in Tourism 2014*, Z. Xiang and I. Tussyadiah, Eds. Cham: Springer International Publishing, 2013, pp. 553–564.

[11] European Commission, Directorate-General for Internal Market, Industry, Entrepreneurship and SMEs, "The European Tourism Indicator System: ETIS toolkit for sustainable destination management," Publications Office of the European Union, Luxembourg, Tech. Rep., 2016.

[12] P. Modica, A. Capocchi, I. Foroni, and M. Zenga, "An assessment of the implementation of the European Tourism Indicator System for sustainable destinations in Italy," *Sustainability*, vol. 10, no. 9, p. 3160, 2018.

[13] M. Fuchs, W. Höpken, and M. Lexhagen, "Big data analytics for knowledge generation in tourism destinations – A case from Sweden," *Journal of Destination Marketing & Management*, vol. 3, no. 4, pp. 198–209, 2014.

[14] J. A. Ivars-Baidal, M. A. Celdrán-Bernabeu, F. Femenia-Serra, J. F. Perles-Ribes, and D. Giner-Sánchez, "Measuring the progress of smart destinations: The use of indicators as a management tool," *Journal of Destination Marketing & Management*, vol. 19, p. 100531, 2021.

[15] J. A. Ivars-Baidal, J. F. Vera-Rebollo, J. Perles-Ribes, F. Femenia-Serra, and M. A. Celdrán-Bernabeu, "Sustainable tourism indicators: what's new within the smart city/destination approach?" *Journal of Sustainable Tourism*, vol. 31, no. 7, pp. 1556–1582, 2023.

[16] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, and K. Tei, "Generative AI for self-adaptive systems: State of the art and research roadmap," *ACM Transactions on Autonomous and Adaptive Systems*, vol. 19, no. 3, pp. 13:1–13:60, 2024.

[17] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[18] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 1482–1494. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00129

[19] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[20] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[21] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[22] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[23] A. Marginéan, J. Bader, S. Chandra, M. Harman, Y. Jia, K. Mao, A. Mols, and A. Scott, "SapFix: Automated end-to-end repair at scale," in *Proc. 41st Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 269–278.

[24] J. Bader, A. Scott, M. Pradel, and S. Chandra, "Getafix: Learning to fix bugs automatically," *Proc. ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 159:1–159:27, 2019.

[25] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. 47th Int. Conf. on Software Engineering (ICSE)*, 2025, pp. 2188–2200.

[26] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin, V. Zhong, C. Xiong, R. Sun, Q. Liu, S. I. Wang, and T. Yu, "Spider 2.0: Evaluating language models on real-world enterprise Text-to-SQL workflows," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.

[27] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[28] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: Instruction-tuning local large language models for data preprocessing," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 8754–8782.

[29] D. Tu, Y. He, W. Cui, S. Ge, H. Zhang, S. Han, D. Zhang, and S. Chaudhuri, "Auto-Validate by-History: Auto-program data quality constraints to validate recurring data pipelines," in *Proc. 29th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD)*, 2023, pp. 4991–5003.

[30] J. Gray, "Why do computers stop and what can be done about it?" *Tandem Technical Report*, no. 85.7, 1986.

[31] D. Oppenheimer, A. Ganapathi, and D. A. Patterson, "Why do Internet services fail, and what can be done about it?" in *Proceedings of the 4th USENIX Symposium on Internet Technologies and Systems (USITS)*, 2003.

[32] D. Yuan, Y. Luo, X. Zhuang, G. R. Rodrigues, X. Zhao, Y. Zhang, P. U. Jain, and M. Stumm, "Simple testing can prevent most critical failures: An analysis of production failures in distributed data-intensive systems," in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014, pp. 249–265.

[33] V. Crescenzi, G. Mecca, and P. Merialdo, "RoadRunner: Towards automatic data extraction from large web sites," in *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2001, pp. 109–118.

[34] A. H. F. Laender, B. A. Ribeiro-Neto, A. S. da Silva, and J. S. Teixeira, "A brief survey of web data extraction tools," *SIGMOD Record*, vol. 31, no. 2, pp. 84–93, 2002.

[35] E. Ferrara, P. De Meo, G. Fiumara, and R. Baumgartner, "Web data extraction, applications and techniques: A survey," *Knowledge-Based Systems*, vol. 70, pp. 301–323, 2014.

[36] N. Kushmerick, "Wrapper verification," *World Wide Web*, vol. 3, no. 2, pp. 79–94, 2000.

[37] P. Vassiliadis, A. Simitsis, and S. Skiadopoulos, "Conceptual modeling for ETL processes," in *Proceedings of the 5th ACM International Workshop on Data Warehousing and OLAP (DOLAP '02)*. McLean, VA, USA: ACM, 2002, pp. 14–21.

[38] P. Vassiliadis, "A survey of Extract–Transform–Load technology," *International Journal of Data Warehousing and Mining*, vol. 5, no. 3, pp. 1–27, 2009.

[39] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001.

[40] M. Stonebraker, D. Bruckner, I. F. Ilyas, G. Beskales, M. Cherniack, S. B. Zdonik, A. Pagan, and S. Xu, "Data curation at scale: The Data Tamer system," in *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6-9, 2013, Online Proceedings*. www.cidrdb.org, 2013. [Online]. Available: http://cidrdb.org/cidr2013/Papers/CIDR13_Paper28.pdf

[41] T. Furche, G. Gottlob, L. Libkin, G. Orsi, and N. W. Paton, "Data wrangling for big data: Challenges and opportunities," in *Proceedings of the 19th International Conference on Extending Database Technology (EDBT 2016)*. Bordeaux, France: OpenProceedings.org, 2016, pp. 473–478.

[42] M. Franklin, A. Halevy, and D. Maier, "From databases to dataspaces: A new abstraction for information management," *ACM SIGMOD Record*, vol. 34, no. 4, pp. 27–33, 2005.

[43] J. Duggan, A. J. Elmore, M. Stonebraker, M. Balazinska, B. Howe, J. Kepner, S. Madden, D. Maier, T. Mattson, and S. B. Zdonik, "The BigDAWG polystore system," *ACM SIGMOD Record*, vol. 44, no. 2, pp. 11–16, 2015.